

AIPL ユーザーズマニュアル

Actor-based Intelligent Parallel Language

児玉靖

yaskodama@gmail.com

2026 年 5 月 13 日

Abstract

AIPL (Actor-based Intelligent Parallel Language; 旧称 ABCL/c+) は, 東京工業大学・米澤明憲研究室で設計された並行オブジェクト指向言語 ABCL/1 を, 現代的なシンタックスとマルチランタイム (OCaml ネイティブ・ブラウザ JS・Node.js・C) で再実装した言語である. 本マニュアルは言語仕様の概要と, 付属サンプルの解説, 組込み関数のリファレンスをまとめる.

後方互換のため, ファイル拡張子 `.abcl`, Python ランタイムのモジュール名 (`aipl_main.py` 等), 環境変数 `ABCL_AI_PROVIDER` などは旧称のまま維持している.

対象バージョン: 本リポジトリ (`abclcp-project`) 同梱の OCaml REPL 実装
主要ソース: `src/lexer.mll`, `src/parser.mly`, `src/eval_thread.ml`
サンプル群: `abclc/*.abcl`, `src/*.abcl`

Contents

1	はじめに	3
2	処理系のビルドと実行	3
2.1	ビルド	3
2.2	サンプル実行	3
2.3	ブラウザ実行	3
3	言語仕様	3
3.1	字句要素	3
3.2	型と値	4
3.3	式と文	4
3.4	クラスとアクター	5
3.5	メッセージ送信	6
3.6	<code>select</code> による選択受信	6
3.7	<code>become</code> による振る舞い更新	7
3.8	<code>reply</code> と <code>sender</code>	7
3.9	リモート送信	7
3.10	レコードと組型 (タプル)	7
4	組込み関数リファレンス	8
4.1	入出力	8
4.2	制御	8
4.3	数学関数	8
4.4	配列	8
4.5	SDL 描画	9
4.6	Web ゲートウェイ	9
4.7	AI 呼び出し (<code>ai_call</code>)	9
4.8	デバッグ	9

5	サンプルプログラム	10
5.1	Hello — 最小サンプル	10
5.2	Counter — 自己メッセージとフィールド	10
5.3	PingPong — 二者間メッセージ往復	11
5.4	Become — 振る舞いの動的入れ替え	11
5.5	Records — レコード型	11
5.6	Tuples — 組型	12
5.7	BoundedBuffer — 生産者・消費者問題	12
5.8	Philosophers — 食事する哲学者	13
5.9	Rotate4Lines — SDL 描画とタイマー	13
5.10	WebCalc — HTTP ゲートウェイと <code>select</code>	13
6	Phase C-E2 型推論 (Python ランタイム)	14
6.1	CLI: <code>--infer</code> と <code>--check</code>	14
6.2	推論される情報	15
6.3	達成された機能 (Phase C → E-2)	15
6.4	<code>where</code> 句の使い方	15
6.5	詳細ドキュメント	16
7	AIPL v2 Distributed Runtime (<code>aipl_dist</code>)	16
7.1	マスター切替	16
7.2	8 機能の一覧 (<code>AIPL_DIST_ENABLE=1</code> 前提)	16
7.3	クイックスタート	16
7.4	PsiLang v3 silent-hang シナリオへの対策一式	17
7.5	サンプル (8 機能 × 3 = 24)	17
7.6	公開 API (<code>aipl_dist</code> モジュール)	17
7.7	進化計算による設計探索の出自	18
8	付録: トラブルシューティング	19

1 はじめに

AIPL は アクターモデルに基づく並行プログラミング言語である。プログラムは複数の独立したアクター (クラスのインスタンス) から構成され, アクター同士は **非同期メッセージ**によって通信する。

- 各アクターは固有のメッセージキュー (メールボックス) を持つ
- 状態 (フィールド) はアクター内に閉じ込められ, 外部から直接読み書きできない
- メソッド呼び出しは **メッセージ送信 (send)** として表現される
- 受信側は到着順, または `select` 文で条件付きに処理する

伝統的な ABCL/1 にあった `past/now/future` の三種類の送信モードを, AIPL では `send (past, fire-and-forget)` / `now expr.m()` (同期返答) / `future expr.m() + await` (非同期返答) として完全に提供する。

2 処理系のビルドと実行

2.1 ビルド

トップディレクトリで `make` を実行すると OCaml 実装と JS 実装の両方をビルドする。

```
make          # = make all (OCaml + JS)
make ocaml    # OCaml REPL のみ (_build/default/src/repl_thread.exe)
make js       # ブラウザ用パーサ再生成
```

2.2 サンプル実行

```
make run-hello      # abcl/Hello.abcl
make run-philosophers # 5 哲学者
make run-rotate4    # SDL 描画デモ
./run_bounded_buffer.sh # 有界バッファ
./run_pingpong_xinu.sh # PingPong (XINU バックエンド)
```

任意のソースを直接食わせるには:

```
_build/default/src/repl_thread.exe abcl/Hello.abcl
```

2.3 ブラウザ実行

`src/browser-abcl/` 以下にブラウザ版がある。`make serve-js` で <http://localhost:3000/> から各デモ HTML (`viz_philosophers.html` 等) が開ける。

3 言語仕様

3.1 字句要素

コメント

```
// 行コメント
/* ブロック
   コメント */
```

キーワード

```
class method var float new become
send send! call remote reply
if then else while do
select case timeout ->
self sender now future await
```

演算子

種別	演算子
算術	+ - * /
比較	== != < <= > >=
代入	=
フィールド/インデックス	. [n]

+ は数値加算と文字列連結の両方に使われる。片方が文字列なら他方は自動で文字列化される ("count = " + 5).

リテラル

- 整数: 0, 42
- 浮動小数: 3.14, 100. (末尾ドット可)
- 文字列: "hello\n" (エスケープは \\, \", \n, \t, \r)
- 識別子: [A-Za-z_] [A-Za-z0-9_]*

3.2 型と値

AIPL は **Hindley-Milner 型推論** を中核に持ちつつ、動的値表現を併用する。代表的な型を以下に示す。

型名	例	備考
int	42	64bit 整数
float	3.14, 100.	倍精度
string	"abc"	UTF-8
bool	比較演算の結果	
actor(C)	new C() の結果	クラス C のインスタンス
array	array_empty() 等	要素は同型
record	{a: 1, b: "x"}	構造的型, フィールド型を保持
tuple	(1, "two", 3.0)	位置別型, 長さ固定
future	future expr.m()	後で await で取り出す
unit	print(...) の戻り	

typeof(x) で実行時に型名 (文字列) を取得できる。

3.3 式と文

式

```

expr := リテラル
| 識別子                      // 変数参照、self、sender
| expr op expr                // 二項演算
| new C(arg, ...)             // アクター生成 (式中)
| f(arg, ...)                  // 組込み関数呼び出し
| { l1: e1, l2: e2, ... }     // レコードリテラル
| ( e1, e2, ... )             // タプル (>=2 要素)
| expr . field                 // フィールドアクセス
| expr [ n ]                   // タプル / 配列インデックス
| now target.m(args)          // 同期メッセージ送信
| future target.m(args)       // 非同期送信 -> future
| await expr                   // future 値を取り出す
| ( expr )

```

文

```

stmt := var x = expr ;         // ローカル変数宣言
| x = expr ;                   // 代入
| f(arg, ...) ;                // 関数呼び出し (戻り値破棄)
| call f(arg, ...) ;           // 同上 (明示形)
| send target.method(args) ;    // 非同期メッセージ送信
| send! target.method(args) ;   // 型検査をバイパスする送信
| become C(args) ;              // 自己の振る舞い置換
| if (expr) stmt [else stmt]
| while expr do stmt
| { stmt; stmt; ... }           // ブロック
| select { case ... timeout ... }

```

if の条件は expr で, ==, !=, <, <=, >, >= を組み合わせる.

3.4 クラスとアクター

```

class ClassName {
  // フィールド宣言 (初期化必須)
  var fieldA = 0;
  var fieldB = 0.0;
  float fieldC = 1.5;    // 型付き宣言 (float のみ専用キーワード)

  method init(args...) { ... }    // インスタンス生成時に自動起動
  method other(args...) { ... }
}

```

- フィールドは **必ず初期化付き**で宣言する
- `init` メソッドが定義されていれば `new` 時に自動で実行される
- 未定義の場合, 生成時のメッセージはスキップされ警告ログが出る

トップレベルで実体化:

```
var name = new ClassName(args);
```

`name` がそのままアクター名 (メールボックス参照) になる.

3.5 メッセージ送信

send — 非同期メッセージ送信 (推奨)

```
send target.method(args);
```

- target はトップレベルで作ったアクター変数か, self / sender
- 呼び出しは即座に戻り, メッセージは相手のメールボックスに積まれる
- 型検査が通ったメッセージのみ送る

send! — 型検査をバイパス

リモート相手やインタプリタが完全に型を見切れない動的状況のためのエスケープハッチ. 通常は `send` を使う.

call — 組込み関数の呼び出し

組込み関数 (`print`, `sdl_*`, `wait` など) に対しては `call f(...)` または単に `f(...)`; を使う. これは戻りを待つ通常の関数呼び出しで, 他アクターのメソッド呼び出しではない.

now / future / await (同期・非同期返答付き送信)

```
var v = now calc.add(3, 4);           // ブロックして reply 値を取得
var f = future ai.ask("long task");   // すぐ Future 値を返す
var ans = await(f);                  // 後で値を取り出す
```

ABCL/1 で言うところの `now` 型送信と `future` 型送信に対応する.

3.6 select による選択受信

メールボックスから 特定のメッセージだけを待ち受けたい場合に使う.

```
select {
  case add(a, b) -> {
    reply(a + b);
  }
  case mul(a, b) -> {
    reply(a * b);
  }
  timeout 15000 -> {
    print("15 秒待っても来なかった");
  }
}
```

- 各 case のパターンは メソッド名 (変数名, ...) だけを書ける (ガード式は無し)
- 一致しないメッセージはキューに残る (消費されない)
- timeout N はミリ秒. N 経過すると対応するブロックが実行される
- timeout 節は省略可 (その場合は永久に待つ)

3.7 become による振る舞い更新

アクター自身を別のクラスに「化ける」ことができる。

```
class A {
  method ping() {
    print("A.ping");
    become B();
  }
}
class B {
  method ping() {
    print("B.ping");
    become A();
  }
}
```

become はメソッド本体内でのみ使え、現在のアクターのフィールドとメソッドを新クラスの init 結果で置き換える。メールボックスは保持される。

3.8 reply と sender

メソッド本体では暗黙の変数 sender が使える (直前にこのメッセージを送ったアクター)。

```
method add(a, b) {
  reply(a + b); // sender に "戻り値" メッセージを送る
}
```

- reply(v) は sender 側に値 v を返す
- 送信側が Web ゲートウェイ (HTTP) 経由なら, HTTP 応答ボディとして返る
- 送信側がアクターなら, 待ちパターン (select) に届く

send sender.X(args) のように sender に対して任意のメソッドを送ることも可能。

3.9 リモート送信

別ホストや別プロセスのアクターには次の構文で送れる。

```
send remote("localhost:8080", "fork2").take(0);
```

- 第 1 引数: ホストとポート (OCaml 側 web_listen で待ち受ける)
- 第 2 引数: 相手プロセス内のアクター名

3.10 レコードと組型 (タプル)

OCaml ランタイムでは構造的なレコードとタプルがリテラルとフィールド/インデックスアクセスとともに利用できる。HM 型推論が型を構造的に保持する。

```
// レコード: ラベル付きフィールド
var alice = {name: "alice", age: 30};
print(alice.name); // "alice"
print(alice.age); // 30
var p = {pos: {x: 3, y: 4}, label: "origin"};
print(p.pos.x); // 3 -- チェーンアクセス可

// 組型 (タプル): 位置別型, >=2 要素
```

```
var t = (1, "two", 3.5);
print(t[0]);           // 1
print(t[1]);           // "two"
print(t[2]);           // 3.5
var pair = ((10, 20), "label");
print(pair[1]);        // "label"
```

型表現:

- レコード: {name : string; age : int}
- 組型: (int * string * float)

両者とも HM 推論で unify 時にラベル/位置と要素型を照合する.

4 組込み関数リファレンス

src/eval_thread.ml の prim_table に登録されている関数群である. すべて `f(args)` または `call f(args);` で呼び出す.

4.1 入出力

関数	説明
<code>print(v)</code>	値を文字列化して標準出力へ. Web UI のログにも残る
<code>typeof(v)</code>	型名 ("int", "float", "string", "actor(C)" ...) を返す

4.2 制御

関数	説明
<code>wait(ms)</code>	現在のアクター (スレッド) を ms ミリ秒スリープ

4.3 数学関数

`sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `sqrt`, `exp`, `log10`, `abs`, `floor`, `ceil`, `round` — いずれも `float` → `float`.

4.4 配列

関数	説明
<code>array_empty()</code>	空配列を生成
<code>array_len(a)</code>	要素数
<code>array_get(a, i)</code>	i 番目の要素 (範囲外は例外)
<code>array_set(a, i, v)</code>	i 番目を v に置換した 新しい配列 を返す
<code>array_push(a, v)</code>	末尾に v を追加した 新しい配列 を返す

配列は永続的データ構造として扱われる (破壊的更新は無い). したがって `a = array_set(a, 0, 99.);` のように再代入する.

4.5 SDL 描画

関数	説明
<code>sdl_init(w, h)</code>	ウィンドウを開く
<code>sdl_clear()</code>	画面クリア
<code>sdl_present()</code>	フロントバッファ反映
<code>sdl_line(x1,y1,x2,y2)</code>	白線
<code>sdl_line_c(x1,y1,x2,y2,r,g,b)</code>	RGB 指定線
<code>sdl_erase_line(x1,y1,x2,y2)</code>	線を消去 (背景色で再描画)
<code>sdl_poll_key()</code>	キーのスキャンコード (押されてなければ 0)
<code>sdl_mouse_x() / sdl_mouse_y()</code>	マウス座標
<code>sdl_mouse_down()</code>	左ボタン状態 (0/1)

4.6 Web ゲートウェイ

関数	説明
<code>web_listen(port)</code>	内蔵 HTTP サーバを起動
<code>web_expose(path, actorName)</code>	フレンドリーパスを公開
<code>reply(v)</code>	HTTP リクエスト発のメッセージなら応答ボディとして返す

POST /api/json/send または POST /api/x/<path> で外部からアクターを呼べる。

4.7 AI 呼び出し (`ai_call`)

```
ai_call("hello")           // 自動選択 (デフォルト Gemini)
ai_call(1, "hello")        // Gemini
ai_call(2, "hello")        // Claude
ai_call(3, "hello")        // ChatGPT
ai_call_with_system(2, "be brief", "hi")
```

第一引数の整数 (省略可) でプロバイダを切り替える:

値	プロバイダ
1 / 省略	Gemini (デフォルト)
2	Anthropic Claude
3	OpenAI ChatGPT

ABCL_AI_PROVIDER=mock を設定すれば全部 mock に流れる (テスト/デモ用).

3 並行モデル経由の AI 呼び出しもそのまま使える:

```
var ans = now ai.ask("hello");           // 同期 reply 待ち
var fut = future ai.ask("long task");     // future 取得
var done = await(fut);                   // 後で取り出す
send ai.ask("fire and forget");           // past 型, 戻り値不要
```

4.8 デバッグ

関数	説明
<code>actor_dump(a)</code>	アクターの型情報 (クラス名・メソッド一覧) を整形出力

5 サンプルプログラム

5.1 Hello — 最小サンプル

abclc/Hello.abcl

```
class Hello {
  var count = 0.;

  method init(n) {
    count = n;
    print("Hello object initialized with " + n);
  }

  method greet() {
    print("Hello! count = " + count);
  }

  method inc() {
    count = count + 1.;
    print("count incremented to " + count);
  }
}

var h = new Hello(5);    // init(5) が自動呼び出し
send h.greet();          // Hello! count = 5
send h.inc();             // count incremented to 6
send h.greet();          // Hello! count = 6
```

学べること: フィールド宣言, init の自動呼び出し, send による非同期メッセージ, 文字列連結 +.

```
make run-hello
```

5.2 Counter — 自己メッセージとフィールド

abclc/counter.abcl

```
class Counter {
  var count = 0.;
  method inc() {
    count = count + 1.;
    print("count:" + count);
    send self.dec(3.);
  }
  method dec(x) {
    count = count - x;
    print(count);
  }
}

var c1 = new Counter();
var c2 = new Counter();
send c1.inc();
send c2.inc();
```

学べること: self を使って自分自身にメッセージを送る, 複数アクターが独立したメールボックスを持つこと.

5.3 PingPong — 二者間メッセージ往復

abclc/PingPong.abcl

```
class Pinger {
  method init() {
    print("Pinger starting");
    send ponger.ping();
  }
  method pong() {
    print("Pinger got pong");
    send sender.ping();
  }
}

class Ponger {
  method ping() {
    print("Ponger got ping");
    send sender.pong();
  }
}

var pinger = new Pinger();
var ponger = new Ponger();
```

学べること: sender による「直前の送信元」への返信, 循環メッセージでずっと走り続けるアクター. init の中から外部アクターへ send できる.

5.4 Become — 振る舞いの動的入れ替え

abclc/become.abcl

```
class A {
  method ping() {
    print("A.ping");
    become B();
  }
}

class B {
  method ping() {
    print("B.ping");
    become A();
  }
}

var x = new A();
send x.ping(); // A.ping -> B 化
send x.ping(); // B.ping -> A 化
send x.ping(); // A.ping
```

学べること: 同じアクター変数 x の振る舞いが受信ごとに切り替わる. メッセージの順序は保たれ, become は次のメッセージから効く.

5.5 Records — レコード型

abclc/Records.abcl (OCaml ランタイムで追加された新機能)

```
var alice = {name: "alice", age: 30};
print("alice.name = " + alice.name);
print("alice.age = " + alice.age);
```

```
// ネストレコードとチェーンアクセス
var p = {pos: {x: 3, y: 4}, label: "origin"};
print("p.label = " + p.label);
print("p.pos.x = " + p.pos.x);
print("p.pos.y = " + p.pos.y);
```

学べること: 構造的レコード型 (TRecord) の生成と参照. HM 推論はラベル名とフィールド型を完全に保持し, unify 時にラベル集合と各型を厳密に照合する.

5.6 Tuples — 組型

abcl/_tuples.abcl

```
var t = (1, "two", 3.5);
print("t[0] = " + t[0]);
print("t[1] = " + t[1]);
print("t[2] = " + t[2]);

var pair = ((10, 20), "label");
print("pair[1] = " + pair[1]);
```

学べること: 位置別型を持つ組型 (TTuple) の作成. インデックスはリテラル整数のみ (静的に位置を解決し, 要素型を取り出す).

5.7 BoundedBuffer — 生産者・消費者問題

abcl/bounded_buffer.abcl (抜粋)

```
class Buffer {
  var cap = 4;
  var s0 = 0; var s1 = 0; var s2 = 0; var s3 = 0;
  var head = 0; var tail = 0; var count = 0;
  var pwaiter = ""; var pitem = 0;
  var cwaiter = "";

  method put(item) {
    if (cwaiter != "") {
      send sender.put_ok();
      send cwaiter.got(item);
      cwaiter = "";
    } else {
      if (count == cap) {
        pwaiter = sender; // 満杯 → 生産者を待たせる
        pitem = item;
      } else {
        // ... スロットに格納 ...
        send sender.put_ok();
      }
    }
  }
}

method get() { /* ... */ }
```

学べること:

- ロックを **使わず**に同期する手法 (待機キューを自前のフィールドに保持)
- sender を保存しておき, 後で send pwaiter.put_ok(); のように「呼び出し側に応答を返す」非同期パターン

- 配列ではなくスロット変数で容量 4 を表現 (言語の最小機能で書ける例)

5.8 Philosophers — 食事する哲学者

abcl/Philosophers5.abcl (5 哲学者). 5 つのアクター間の対称デッドロックを, ハンドオフによって回避する古典問題. SDL 版 (Philosophers5Gui.abcl) ではビジュアライズも可能.

```
make run-philosophers      # コンソール
./run_viz_philosophers.sh # SDL ビジュアル
```

5.9 Rotate4Lines — SDL 描画とタイマー

abcl/Rotate4Lines.abcl

```
class Line {
  var cx = 0.0; var cy = 0.0;
  var angle = 0.0; var len = 50.0;
  var r = 255; var g = 255; var b = 255;
  var x1 = 0.0; var y1 = 0.0; var x2 = 0.0; var y2 = 0.0;
  var drawn = 0;

  method init(startCx, startCy, startAngle, cr, cg, cb) {
    cx = startCx; cy = startCy; angle = startAngle;
    r = cr; g = cg; b = cb;
  }

  method rotate() {
    if (drawn == 1) { call sdl_erase_line(x1, y1, x2, y2); }
    angle = angle + 3.0;
    var rad = angle * 3.14159 / 180.0;
    var dx = cos(rad) * len;
    var dy = sin(rad) * len;
    x1 = cx - dx; y1 = cy - dy;
    x2 = cx + dx; y2 = cy + dy;
    call sdl_line_c(x1, y1, x2, y2, r, g, b);
    call sdl_present();
    drawn = 1;
    call wait(32);
    send self.rotate();
  }
}

sdl_init(500, 500);
var li1 = new Line(125.0, 125.0, 0.0, 255, 80, 80);
var li2 = new Line(375.0, 125.0, 90.0, 80, 255, 120);
var li3 = new Line(125.0, 375.0, 180.0, 80, 160, 255);
var li4 = new Line(375.0, 375.0, 270.0, 255, 200, 40);
send li1.rotate(); send li2.rotate(); send li3.rotate(); send li4.rotate();
```

学べること: wait(ms) と send self.rotate() の組み合わせによる「自前のタイマーループ」, 4 つのアクターが独立 30FPS で回るマルチスレッド描画.

5.10 WebCalc — HTTP ゲートウェイと select

src/web_calc1.abcl

```
class Calc {
  method init() {
    print("Calc initialized");
```

```

    send self.main();
}

method add(a, b) {
    reply(999);          // すぐ来た add は 999 を返す
}

method main() {
    print("waiting...");
    select {
        case add(a, b) -> {
            reply(a + b);    // main 待ちの間に来た add だけ正規計算
        }
        timeout 15000 -> {
            print("timeout occurred");
        }
    }
    print("select finished");
}
}

var calc = new Calc();
web_listen(8080);
web_expose("/calc", "calc");
print("Open http://localhost:8080/ and send to actor 'calc'");

```

学べること:

- select で「特定のメッセージだけ」を待つ書き方
- timeout 節
- reply で HTTP 応答に値を返す
- web_listen + web_expose による外部 API 化

ブラウザで `http://localhost:8080/` を開き, actor calc に `{"method":"add","args":[3,4]}` を POST すると 7 が返ってくる.

6 Phase C-E2 型推論 (Python ランタイム)

Phase 11 系の gradual 静的型検査に加え, Python 版 AIPL は Phase C 以降で **constraint-based Hindley–Milner 型推論 + Z3 refinement** を備える. 注釈の明示が無くてもプログラム全体の型情報を組み立て, 後段の Phase E-2 統合 CLI (`--check`) で type-check と inference を同時に走らせられる.

実装ファイル: `src/python-aipl/aipl_inference.py` (約 1255 LOC, Phase E-2 時点).

6.1 CLI: `--infer` と `--check`

フラグ	役割
<code>--type-check</code> <code>--infer</code>	Phase 11 系の signature-based gradual checker (既存) constraint-based HM 推論 + Z3 refinement を走らせ, メソッド毎に推論結果を表示して終了
<code>--check</code>	上記 2 つを section ヘッダ付きで 同時実行 (Phase E-2). 両方の結果を 1 コマンドで得る
<code>--strict</code>	issue があれば exit code 2 (typeck) / 3 (infer) で停止

```
# 型推論のみ
python3 src/python-aipl/aipl_main.py program.aipl --infer

# 統合: type-check + 型推論 (推奨)
python3 src/python-aipl/aipl_main.py program.aipl --check

# 厳密モード: いずれかが issue を出せば exit
python3 src/python-aipl/aipl_main.py program.aipl --check --strict
```

--infer / --check はプログラムを実行しない (パース後に解析だけ走らせて exit).

6.2 推論される情報

--infer の出力例 (feature_b_crossclass/sample1_simple.aipl より):

```
=== Adder.add ===
  params:
    x : Int
    y : Int
  return : Int

=== Bridge.use_adder ===
  params:
    other : Adder
    a : Int
    b : Int
  return : Int
  locals:
    s : Int

[infer] 2 method(s), 0 unify issue(s), 0 refinement issue(s)
```

注釈ゼロのコードから Adder.add の $\text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$ と Bridge.use_adder の other : Adder までをクラス越境で逆推論している.

6.3 達成された機能 (Phase C $\rightarrow$ E-2)

機能	Phase	説明
基本 HM 型推論	C	$\text{Int} \rightarrow \text{Int}$ の関数や Bool 述語, Rat/Real の自動推論
Cross-class 推論	D-1	now obj.method() 経由で型がクラス間を流れる
where 句 (refinement)	E- α	$\text{Int where } k \geq 0$ を AIPL 表層で記述, Z3 検査
Actor field 共有	E- β	<code>var n = 0;</code> のクラス field を method 横断で 1 TVar に固定
Record structural	E- γ	{a: Int, b: Str} 構造的型 + width subtyping
Real/Rat refinement	E- γ -R	$\text{Real where } x > 0.0$ の Z3 Real 理論判定
typeck $\times$ inference 統合	E-2	--check で 1 コマンド, BUILTIN_SIGNATURES 共有

6.4 where 句の使い方

```
class Engine {
  method process(r: Int where r >= 0 and r <= 100) -> Int {
    reply(r * 2);
  }
}
```

AIPL_v2_TypeInference 仕様で進化計算により発見された設計. 充足不可能な refinement (vacuous false) は declaration-time に検査される:

```
method bad(k: Int where k >= 5 and k <= 3) -> Int { reply(k); }
// -> refinement issue: vacuously false (Z3 unsat)
```

6.5 詳細ドキュメント

各 Phase の設計記録は `aice-pi-evolution/experiments/2026-05-17_aipl_v2_type_inference/`: `PHASE_C_REPORT.md` (HM + Z3), `PHASE_D_REPORT.md` (cross-class), `PHASE_E_REPORT.md` (where 句), `PHASE_E_BETA_REPORT.md` (actor field), `PHASE_E_GAMMA_REPORT.md` (record structural), `PHASE_E_GAMMA_R_REPORT.md` (Real/Rat), `PHASE_E_2_REPORT.md` (--check 統合), `SAMPLES_SNAPSHOT.md` (7 feature × 3 = 21 sample の実行スナップショット).

7 AIPL v2 Distributed Runtime (aipl_dist)

`aipl_dist.py` は AIPL v2 (2) “Distributed” の進化計算で発見された設計 (I0003 balanced / I0023 hang-resilience / I0036 Erlang OTP の 3 候補) を ランタイム層に **opt-in** で重ね合わせるモジュール (約 591 LOC). AIPL 言語仕様は不変, 既存サンプルは無改変で同じ挙動.

実装ファイル: `src/python-aipl/aipl_dist.py` (新規モジュール), `aipl_interp.py` (spawn 時の自動 hook, +27 行), `aipl_runtime.py` (actor 失敗時の自動 hook, +13 行).

7.1 マスター切替

```
export AIPL_DIST_ENABLE=1 # これが unset / "0" なら全機能 no-op
```

`AIPL_DIST_ENABLE=1` を立てないと, `aipl_dist` の関数群は **すべて即 None / False** を返す **no-op**. 既存プログラムへの影響ゼロを構造的に保証する.

7.2 8 機能の一覧 (AIPL_DIST_ENABLE=1 前提)

機能	env var	説明
I-1 <code>env_var_routing</code>	<code>AIPL_ROUTE="Name1:tag1,..."</code>	actor 名 → tag のルーティングテーブル
I-2 <code>structured_log</code>	<code>AIPL_DIST_LOG_FILE=/path</code>	ND-JSON 1 行 / event, スレッドセーフ書込
I-3 <code>token_budget_aware</code>	<code>AIPL_DIST_RPM / AIPL_DIST_TPM</code>	60 秒スライディング窓で AI call をレート制限
I-4 <code>checkpoint_and_resume</code>	<code>AIPL_DIST_CHECKPOINT_DIR=/path</code>	actor field を atomic に save, spawn 時 restore
IQ <code>quarantine_and_skip</code>	<code>AIPL_DIST_QUARANTINE_TTL=60</code>	actor 失敗時に自動隔離, 後続 send は skip
IM-1 <code>quorum_replicate</code>	<code>AIPL_DIST_QUORUM_PROVIDERS=".."</code>	並列で N provider に投げ, 最初に応答したものを採用
IM-2 <code>subtree_quarantine</code>	<code>AIPL_DIST_SUBTREE_QUARANTINE=1</code>	actor 失敗時にその子孫も一括隔離 (Erlang OTP 風)
integration	(上記の組合せ)	spawn ツリー追跡 + 観測性 + レジリエンス

7.3 クイックスタート

`AIPL_DIST_ENABLE=1` 一発で観測性 (I-2) と spawn ログだけが ON になる:


```

AIPL_AI_PROVIDER=mock AIPL_DIST_ENABLE=1 \
  AIPL_DIST_LOG_FILE=/tmp/aip1.ndjson \
  python3 src/python-aip1/aip1_main.py program.aip1

cat /tmp/aip1.ndjson
# {"ts": ..., "event": "actor_spawn", "actor": "counter", "cls": "Counter"}
# {"ts": ..., "event": "actor_routed", "actor": "counter", ...}

```

7.4 PsiLang v3 silent-hang シナリオへの対策一式

PsiLang v3 trial #1 で OpenAI silent hang により 35/38 個体喪失した問題に対し,3 MVP を同時有効化すると **call-level / actor-level / subtree-level** の **3 重防御**:

```

export AIPL_DIST_ENABLE=1
export AIPL_DIST_QUORUM_PROVIDERS="openai,anthropic,gemini" # IM-1
export AIPL_DIST_QUARANTINE_TTL=60 # IQ
export AIPL_DIST_SUBTREE_QUARANTINE=1 # IM-2
export AIPL_DIST_CHECKPOINT_DIR=/tmp/aip1_ck # I-4
export AIPL_DIST_LOG_FILE=/tmp/aip1.ndjson # I-2

```

- **call-level**: OpenAI が hang しても anthropic/gemini が応答 → caller 継続
- **actor-level**: それでも actor が落ちたら自動 quarantine (60s)
- **subtree-level**: 子孫 actor も一括隔離 (Erlang OTP blast-radius containment)
- **persistent state**: actor 再 spawn 時にチェックポイント復元
- **observability**: 全イベントが NDJSON で記録

7.5 サンプル (8 機能 × 3 = 24)

aiice-pi-evolution/experiments/2026-05-17_aip1_v2_type_inference/IMPL_I0003_MVP/samples/
配下に feature 別サブディレクトリで整理:

```

samples/
i1_env_var_routing/      sample1_basic.aip1, sample2_no_match.py, sample3_log_correlation.py
i2_structured_log/      sample1_basic.py, sample2_no_file.py, sample3_multi_thread.py
i3_token_budget/        sample1_basic.py, sample2_blocking.py, sample3_aip1_integration.
  aip1
i4_checkpoint/          sample1_basic.py, sample2_atomic.py, sample3_list_states.py
iq_quarantine/          sample1_basic.aip1, sample2_ttl_expiry.py, sample3_manual_clear.py
im1_quorum/             sample1_first_wins.py, sample2_tolerates_one_failure.py,
  sample3_all_fail.py
im2_subtree_quarantine/ sample1_basic.aip1, sample2_grandchildren.aip1,
  sample3_unrelated_unaffected.aip1
integration/            sample1_basic.aip1, sample2_combined_logging.aip1,
  sample3_full_resilience.aip1

```

ユニットテストは IMPL_I0003_MVP/tests/test_aip1_dist.py (27 個).

7.6 公開 API (aip1_dist モジュール)

```

import aip1_dist

# Master toggle
aip1_dist.is_enabled() # -> bool

# I-1 routing

```

```

aipl_dist.route_for(name)          # -> Optional[str]
aipl_dist.parse_route_table(raw)    # -> dict[str, str]

# I-2 log
aipl_dist.log_event(event, **fields) # -> bool

# I-3 budget
aipl_dist.token_budget_gate()       # -> Optional[TokenBudgetGate]
aipl_dist.call_ai_with_budget(prompt, call_ai_fn=None, **kw)

# I-4 checkpoint
aipl_dist.save_actor_state(name, state) # -> bool
aipl_dist.restore_actor_state(name)     # -> Optional[dict]
aipl_dist.list_actor_states()           # -> dict[str, str]

# IQ quarantine
aipl_dist.quarantine_actor(name, ttl=None) # -> bool
aipl_dist.is_quarantined(name)            # -> bool
aipl_dist.clear_quarantine(name)          # -> bool
aipl_dist.quarantine_status()             # -> dict[str, float]

# IM Erlang OTP
aipl_dist.register_spawn(child, parent)
aipl_dist.descendants_of(actor)            # -> list[str]
aipl_dist.quarantine_subtree(actor, ttl=None)
aipl_dist.call_ai_quorum(prompt, providers=None, **kw)

```

すべての関数は AIPL_DIST_ENABLE != "1" 時に None / False / 空 dict を返す no-op.

7.7 進化計算による設計探索の出自

aipl_dist.py の設計は手書きではなく, AIPL_v2_Distributed.aice (11 軸 GA 仕様) を MAP-Elites で 8 seed × 30 gen 探索した結果から派生:

- 仕様: AIPL_v2_Distributed.aice / .ga.json / .schema.json
- Run 1 / Run 2 結果: distributed_run_outputs/ (38 個体 / 25 cells / 24 min × 2)
- 設計レポート: IMPL_DESIGN.md, IMPL_RUN_REPORT.md, IMPL_INTEGRATION_REPORT.md, IMPL_I0023_REPORT.md, IMPL_I0036_REPORT.md

8 付録: トラブルシューティング

症状	対処
Unknown function: foo	組込みの綴り誤り. <code>prim_table (src/eval_thread.ml)</code> の登録名と照合
Actor X not found	<code>var X = new ...</code> ; より前に <code>send X....</code> していないか確認
[Actor] X.init arity mismatch	<code>new C(args)</code> の引数数と <code>init</code> の仮引数数が違う
<code>select</code> がいつまでも返らない	パターンが受信メッセージと一致していない/ <code>timeout</code> 未指定
SDL ウィンドウが反応しない	<code>sdl_present()</code> を毎フレーム呼ぶ/ <code>wait</code> で <code>yield</code> する
HTTP リクエストでハング	<code>reply</code> を呼んでいない or <code>select</code> が適切な case を持たない
no overload of foo matches (...)	HM 推論がオーバーロード候補と整合しなかった. 実引数の型を <code>typeof</code> で確認
no field f in record {...}	レコードにそのラベルが無い. ラベル名の綴りを確認
tuple index N out of bounds	組型のサイズと [N] の整合性を確認
[infer] N unify issue(s)	--infer 出力. 位置 (at assign to s 等) を見て注釈を調整
refinement is vacuously false	where 句が Z3 で unsat. 値域を見直す
[actor X.Y] error: ...	実行時 actor 内エラー. <code>AIPL_DIST_ENABLE=1</code> <code>AIPL_DIST_QUARANTINE_TTL=60</code> で自動隔離
AIPL ランタイムが silent hang	<code>AIPL_DIST_QUORUM_PROVIDERS="openai,anthropic"</code> で並列フェイルオーバ
OpenAI tier-1 rate limit	<code>AIPL_DIST_RPM=400</code> <code>AIPL_DIST_TPM=180000</code> で I-3 ゲートを有効化

設計指針のまとめ: AIPL では「同期したいなら sender に `reply`, 待ちたいなら `select`, 状態を切り替えたいなら `become`」と覚えておくと, 大半の並行パターンを言語機能だけで表現できる. 型は HM 推論で自動的に決まるため, 通常は注釈を書かずに走らせて構わない. 明示したい場合のみ Python (annotated) ランタイムを使うとよい.